

UNIVERSIDAD PERUANA UNIÓN

FACULTAD DE INGENIERÍA Y ARQUITECTURA

E.P. Ingeniería de Sistemas

TRABAJO PRÁCTICO: BASE DE DATOS II

Apellidos y Nombres:	Equipo ERP Agro Peruvian Foods S.A.C.
Curso:	Base de Datos 2
Ciclo / Grupo:	Ciclo 4 Grupo 2 (Tríada UPeU: ADS BD2 LP2)
Proyecto / Esquema:	ERP Agro Peruvian Foods S.A.C. (BOMERP_APP)
Plataforma SGBD / Herramienta:	Oracle Database 23c Free / Oracle SQL Developer
Fecha:	18 de Septiembre de 2026

1. Modelo Relacional del Proyecto

A continuación se presenta el diagrama del modelo relacional integral para el sistema **ERP Agro Peruvian Foods S.A.C.** implementado en Oracle Database, que interconecta el catálogo, compras, caja chica, órdenes de producción con trazabilidad FEFO y el módulo de ventas y facturación.



[PEGAR AQUÍ CAPTURA DEL MODELO RELACIONAL COMPLETO DE SQL DEVELOPER / DATA MODELER]

Tablas requeridas: CATEGORIAS, PRODUCTOS, LOTES_PRODUCTO, PROVEEDORES, COMPRAS, DETALLE_COMPRAS, SESIONES_CAJA, MOVIMIENTOS_CAJA, ORDENES_PRODUCCION, DETALLES_CONSUMO_INSUMO, VENTAS, DETALLE_VENTAS.

2. Implementación de la Base de Datos y Datos de Prueba

A continuación se adjunta el código SQL exacto ejecutado en Oracle SQL Developer para la creación de las tablas troncales con sus respectivas restricciones de integridad de dominio, claves primarias por identidad y claves foráneas en cascada.

2.1 Código SQL de Implementación DDL (Tablas en Oracle SQL Developer)

```
-- =====
-- 1. TABLAS DE CATÁLOGO E INVENTARIO
-- =====
CREATE TABLE CATEGORIAS (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    NOMBRE VARCHAR2(80 BYTE) NOT NULL CONSTRAINT UQ_CATEGORIA_NOMBRE UNIQUE,
    DESCRIPCION VARCHAR2(200 BYTE)
);

CREATE TABLE PRODUCTOS (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    NOMBRE VARCHAR2(120 BYTE) NOT NULL,
    PRECIO NUMBER(10, 2) NOT NULL CONSTRAINT CK_PROD_PRECIO CHECK (PRECIO > 0),
    STOCK NUMBER(10) DEFAULT 0 NOT NULL CONSTRAINT CK_PROD_STOCK CHECK (STOCK >= 0),
    ID_CATEGORIA NUMBER,
    CONSTRAINT FK_PROD_CATEGORIA FOREIGN KEY (ID_CATEGORIA) REFERENCES CATEGORIAS(ID)
);

CREATE INDEX IDX_PROD_CATEGORIA ON PRODUCTOS(ID_CATEGORIA);
CREATE INDEX IDX_PROD_NOMBRE ON PRODUCTOS(NOMBRE);

-- =====
-- 2. TABLAS DE COMPRAS Y PROVEEDORES
-- =====
CREATE TABLE PROVEEDORES (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    RUC VARCHAR2(11 BYTE) NOT NULL CONSTRAINT UQ_PROV_RUC UNIQUE,
    RAZON_SOCIAL VARCHAR2(150 BYTE) NOT NULL,
    TELEFONO VARCHAR2(20 BYTE),
    EMAIL VARCHAR2(80 BYTE),
    DIRECCION VARCHAR2(200 BYTE)
);

CREATE TABLE COMPRAS (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    NUMERO_COMPROBANTE VARCHAR2(30 BYTE) NOT NULL,
    ID_PROVEEDOR NUMBER NOT NULL,
    FECHA_EMISION DATE DEFAULT SYSDATE NOT NULL,
    ESTADO VARCHAR2(20 BYTE) DEFAULT 'REGISTRADA' NOT NULL
        CONSTRAINT CK_COMPRA_ESTADO CHECK (ESTADO IN ('REGISTRADA', 'PAGADA', 'ANULADA')),
    SUBTOTAL NUMBER(12, 2) NOT NULL,
    IGV NUMBER(12, 2) NOT NULL,
    TOTAL NUMBER(12, 2) NOT NULL CONSTRAINT CK_COMPRA_TOTAL CHECK (TOTAL >= 0),
    SALDO_PENDIENTE NUMBER(12, 2) DEFAULT 0 NOT NULL CONSTRAINT CK_COMPRA_SALDO CHECK (SALDO_PENDIENTE >= 0),
    CONSTRAINT FK_COMPRA_PROV FOREIGN KEY (ID_PROVEEDOR) REFERENCES PROVEEDORES(ID)
);

CREATE TABLE DETALLE_COMPRAS (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ID_COMPRA NUMBER NOT NULL,
    ID_INSUMO NUMBER NOT NULL,
    NOMBRE_INSUMO VARCHAR2(120 BYTE) NOT NULL,
    CANTIDAD NUMBER(12, 3) NOT NULL CONSTRAINT CK_DET_COMPRA_CANT CHECK (CANTIDAD > 0),
    PRECIO_UNITARIO NUMBER(10, 2) NOT NULL CONSTRAINT CK_DET_COMPRA_PRECIO CHECK (PRECIO_UNITARIO > 0),
    SUBTOTAL NUMBER(12, 2) NOT NULL,
    CONSTRAINT FK_DET_COMPRA_CABECERA FOREIGN KEY (ID_COMPRA) REFERENCES COMPRAS(ID) ON DELETE CASCADE
);

-- =====
```

```

-- 3. TABLAS DE CAJA CHICA Y MOVIMIENTOS
-- =====
CREATE TABLE SESIONES_CAJA (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    CAJERO_NOMBRE VARCHAR2(100 BYTE) NOT NULL,
    FECHA_APERTURA TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    FECHA_CIERRE TIMESTAMP,
    MONTO_APERTURA NUMBER(12, 2) NOT NULL CONSTRAINT CK_CAJA_APERTURA CHECK (MONTO_APERTURA >= 0),
    MONTO_CIERRE NUMBER(12, 2),
    SALDO_TEORICO NUMBER(12, 2) NOT NULL,
    SALDO_REAL NUMBER(12, 2),
    DIFERENCIA NUMBER(12, 2),
    ESTADO VARCHAR2(20 BYTE) DEFAULT 'ABIERTA' NOT NULL
        CONSTRAINT CK_CAJA_ESTADO CHECK (ESTADO IN ('ABIERTA', 'CERRADA')),
    OBSERVACIONES VARCHAR2(300 BYTE)
);

CREATE TABLE MOVIMIENTOS_CAJA (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ID_SESION_CAJA NUMBER NOT NULL,
    TIPO VARCHAR2(20 BYTE) NOT NULL CONSTRAINT CK_MOV_TIPO CHECK (TIPO IN ('INGRESO', 'EGRESO')),
    MONTO NUMBER(12, 2) NOT NULL CONSTRAINT CK_MOV_MONTO CHECK (MONTO > 0),
    CONCEPTO VARCHAR2(200 BYTE) NOT NULL,
    METODO_PAGO VARCHAR2(30 BYTE) NOT NULL
        CONSTRAINT CK_MOV_PAGO CHECK (METODO_PAGO IN ('EFECTIVO', 'YAPE', 'PLIN', 'TRANSFERENCIA')),
    REFERENCIA VARCHAR2(100 BYTE),
    FECHA TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT FK_MOV_SESION_CAJA FOREIGN KEY (ID_SESION_CAJA) REFERENCES SESIONES_CAJA(ID)
);

-- =====
-- 4. TABLAS DE PRODUCCIÓN Y LOTES FEFO
-- =====
CREATE TABLE ORDENES_PRODUCCION (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    CODIGO_ORDEN VARCHAR2(30 BYTE) NOT NULL CONSTRAINT UQ_OP_CODIGO UNIQUE,
    ID_PRODUCTO NUMBER NOT NULL,
    NOMBRE_PRODUCTO VARCHAR2(120 BYTE) NOT NULL,
    CANTIDAD_PROGRAMADA NUMBER(10) NOT NULL CONSTRAINT CK_OP_CANT_PROG CHECK (CANTIDAD_PROGRAMADA > 0),
    CANTIDAD_PRODUCIDA NUMBER(10),
    FECHA_INICIO TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    FECHA_FIN TIMESTAMP,
    ESTADO VARCHAR2(20 BYTE) DEFAULT 'PLANIFICADA' NOT NULL
        CONSTRAINT CK_OP_ESTADO CHECK (ESTADO IN ('PLANIFICADA', 'EN_PROCESO', 'COMPLETADA', 'CANCELADA')),
    OBSERVACIONES VARCHAR2(300 BYTE),
    CONSTRAINT FK_OP_PRODUCTO FOREIGN KEY (ID_PRODUCTO) REFERENCES PRODUCTOS(ID)
);

CREATE TABLE DETALLES_CONSUMO_INSUMO (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ID_ORDEN_PRODUCCION NUMBER NOT NULL,
    ID_INSUMO NUMBER NOT NULL,
    NOMBRE_INSUMO VARCHAR2(120 BYTE) NOT NULL,
    CANTIDAD_REQUERIDA NUMBER(12, 3) NOT NULL CONSTRAINT CK_DET_REQ CHECK (CANTIDAD_REQUERIDA > 0),
    CANTIDAD_CONSUMIDA NUMBER(12, 3) DEFAULT 0 NOT NULL,
    UNIDAD_MEDIDA VARCHAR2(20 BYTE) NOT NULL,
    CONSTRAINT FK_DET_OP FOREIGN KEY (ID_ORDEN_PRODUCCION) REFERENCES ORDENES_PRODUCCION(ID) ON DELETE CASCADE
);

CREATE TABLE LOTES_PRODUCTO (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    CODIGO_LOTE VARCHAR2(30 BYTE) NOT NULL CONSTRAINT UQ_LOTE_CODIGO UNIQUE,

```

```

ID_PRODUCTO NUMBER NOT NULL,
NOMBRE_PRODUCTO VARCHAR2(120 BYTE) NOT NULL,
STOCK_DISPONIBLE NUMBER(10) NOT NULL CONSTRAINT CK_LOTE_STOCK CHECK (STOCK_DISPONIBLE >= 0),
FECHA_PRODUCCION DATE NOT NULL,
FECHA_VENCIMIENTO DATE NOT NULL,
CONSTRAINT FK_LOTE_PRODUCTO FOREIGN KEY (ID_PRODUCTO) REFERENCES PRODUCTOS(ID),
CONSTRAINT CK_LOTE_FECHAS CHECK (FECHA_VENCIMIENTO >= FECHA_PRODUCCION)
);

CREATE INDEX IDX_LOTE_FEFO ON LOTES_PRODUCTO(ID_PRODUCTO, FECHA_VENCIMIENTO ASC);

-- =====
-- 5. TABLAS DE VENTAS (CABECERA - DETALLE)
-- =====
CREATE TABLE VENTAS (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    NUMERO_TICKET VARCHAR2(30 BYTE) NOT NULL CONSTRAINT UQ_VENTA_TICKET UNIQUE,
    FECHA TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    ESTADO VARCHAR2(20 BYTE) DEFAULT 'REGISTRADA' NOT NULL
        CONSTRAINT CK_VENTA_ESTADO CHECK (ESTADO IN ('REGISTRADA', 'ANULADA')),
    METODO_PAGO VARCHAR2(30 BYTE) NOT NULL
        CONSTRAINT CK_VENTA_PAGO CHECK (METODO_PAGO IN ('EFFECTIVO', 'YAPE', 'PLIN', 'TRANSFERENCIA')),
    SUBTOTAL NUMBER(12, 2) NOT NULL,
    IGV NUMBER(12, 2) NOT NULL,
    TOTAL NUMBER(12, 2) NOT NULL CONSTRAINT CK_VENTA_TOTAL CHECK (TOTAL >= 0),
    CLIENTE_NOMBRE VARCHAR2(120 BYTE)
);

CREATE TABLE DETALLE_VENTAS (
    ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ID_VENTA NUMBER NOT NULL,
    ID_PRODUCTO NUMBER NOT NULL,
    ID_LOTE NUMBER,
    NOMBRE_PRODUCTO VARCHAR2(120 BYTE) NOT NULL,
    PRECIO_UNITARIO NUMBER(10, 2) NOT NULL CONSTRAINT CK_DET_VNT_PRECIO CHECK (PRECIO_UNITARIO > 0),
    CANTIDAD NUMBER(10) NOT NULL CONSTRAINT CK_DET_VNT_CANT CHECK (CANTIDAD > 0),
    SUBTOTAL NUMBER(12, 2) NOT NULL,
    CONSTRAINT FK_DET_VENTA FOREIGN KEY (ID_VENTA) REFERENCES VENTAS(ID) ON DELETE CASCADE,
    CONSTRAINT FK_DET_VENTA_PROD FOREIGN KEY (ID_PRODUCTO) REFERENCES PRODUCTOS(ID),
    CONSTRAINT FK_DET_VENTA_LOTE FOREIGN KEY (ID_LOTE) REFERENCES LOTES_PRODUCTO(ID)
);

```



[PEGAR AQUÍ CAPTURA DE COMPILACIÓN DDL EN SQL DEVELOPER]

Captura mostrando la ejecución en el SQL Worksheet con las respuestas "Tabla creada" para todas las entidades.

2.2 Código SQL de Poblado de Datos de Prueba (INSERT INTO)

```
-- 1. CATEGORÍAS
INSERT INTO CATEGORIAS (ID, NOMBRE, DESCRIPCION) VALUES (1, 'Granos Andinos', 'Granos nativos orgánicos: quinoa, amaranto, mijo, etc.')
INSERT INTO CATEGORIAS (ID, NOMBRE, DESCRIPCION) VALUES (2, 'Superfoods y Frutas', 'Frutas deshidratadas y frutos secos')
INSERT INTO CATEGORIAS (ID, NOMBRE, DESCRIPCION) VALUES (3, 'Harinas y Polvos', 'Harinas nutritivas procesadas')
INSERT INTO CATEGORIAS (ID, NOMBRE, DESCRIPCION) VALUES (4, 'Insumos y Envases', 'Materiales de empaque, botellas, etc.')

-- 2. PRODUCTOS
INSERT INTO PRODUCTOS (ID, NOMBRE, PRECIO, STOCK, ID_CATEGORIA) VALUES (1, 'Quinua Blanca Real Orgánica 1kg', 14.50, 10, 1)
INSERT INTO PRODUCTOS (ID, NOMBRE, PRECIO, STOCK, ID_CATEGORIA) VALUES (2, 'Quinua Roja Seleccionada 1kg', 15.00, 10, 1)
INSERT INTO PRODUCTOS (ID, NOMBRE, PRECIO, STOCK, ID_CATEGORIA) VALUES (3, 'Kiwicha Perlada en Grano 1kg', 12.00, 10, 1)
INSERT INTO PRODUCTOS (ID, NOMBRE, PRECIO, STOCK, ID_CATEGORIA) VALUES (5, 'Palta Hass Calibre 16 Exportación 1kg', 18.00, 10, 2)
INSERT INTO PRODUCTOS (ID, NOMBRE, PRECIO, STOCK, ID_CATEGORIA) VALUES (6, 'Arándano Fresco Biloxi Clamshell 125g', 6.20, 17, 2)
INSERT INTO PRODUCTOS (ID, NOMBRE, PRECIO, STOCK, ID_CATEGORIA) VALUES (7, 'Cacao Nativo Chuncho en Grano 1kg', 10.00, 10, 3)
INSERT INTO PRODUCTOS (ID, NOMBRE, PRECIO, STOCK, ID_CATEGORIA) VALUES (9, 'Harina de Lúcumá Pura 250g', 18.00, 10, 3)
INSERT INTO PRODUCTOS (ID, NOMBRE, PRECIO, STOCK, ID_CATEGORIA) VALUES (10, 'Maca Negra Gelatinizada en Polvo 250g', 15.00, 10, 3)
INSERT INTO PRODUCTOS (ID, NOMBRE, PRECIO, STOCK, ID_CATEGORIA) VALUES (11, 'Bolsa Bilaminada al Vacío 1kg', 25.00, 10, 4)
INSERT INTO PRODUCTOS (ID, NOMBRE, PRECIO, STOCK, ID_CATEGORIA) VALUES (12, 'Caja de Cartón Corrugado Exportación 1kg', 20.00, 10, 4)

-- 3. PROVEEDORES
INSERT INTO PROVEEDORES (ID, RUC, RAZON_SOCIAL, TELEFONO, EMAIL, DIRECCION) VALUES (1, '20601234567', 'Cooperativa Andina', '0912345678', 'info@cooperativaandina.com', 'Calle Principal 123, Quito')
INSERT INTO PROVEEDORES (ID, RUC, RAZON_SOCIAL, TELEFONO, EMAIL, DIRECCION) VALUES (2, '20558974123', 'Empacadora Frutas y Verduras', '0987654321', 'ventas@empacadora.com', 'Calle Industrial 456, Guayaquil')

-- 4. LOTES (CONTROL DE CADUCIDAD FEFO)
INSERT INTO LOTES_PRODUCTO (ID, CODIGO_LOTE, ID_PRODUCTO, NOMBRE_PRODUCTO, STOCK_DISPONIBLE, FECHA_PRODUCCION, FECHA_EXPIRACION)
VALUES (1, 'LOT-QNB-2026-01', 1, 'Quinua Blanca Real Orgánica 1kg', 350, DATE '2026-07-10', DATE '2027-07-10')
INSERT INTO LOTES_PRODUCTO (ID, CODIGO_LOTE, ID_PRODUCTO, NOMBRE_PRODUCTO, STOCK_DISPONIBLE, FECHA_PRODUCCION, FECHA_EXPIRACION)
VALUES (2, 'LOT-QNB-2026-02', 1, 'Quinua Blanca Real Orgánica 1kg', 500, DATE '2026-09-01', DATE '2027-09-01')
INSERT INTO LOTES_PRODUCTO (ID, CODIGO_LOTE, ID_PRODUCTO, NOMBRE_PRODUCTO, STOCK_DISPONIBLE, FECHA_PRODUCCION, FECHA_EXPIRACION)
VALUES (3, 'LOT-ARA-2026-01', 6, 'Arándano Fresco Biloxi Clamshell 125g', 900, DATE '2026-09-10', DATE '2026-12-10')
INSERT INTO LOTES_PRODUCTO (ID, CODIGO_LOTE, ID_PRODUCTO, NOMBRE_PRODUCTO, STOCK_DISPONIBLE, FECHA_PRODUCCION, FECHA_EXPIRACION)
VALUES (4, 'LOT-PLT-2026-01', 5, 'Palta Hass Calibre 16 Exportación (kg)', 1200, DATE '2026-09-12', DATE '2026-12-12')

-- 5. CAJA CHICA Y TURNOS
INSERT INTO SESIONES_CAJA (ID, CAJERO_NOMBRE, FECHA_APERTURA, MONTO_APERTURA, SALDO_TEORICO, ESTADO, OBSERVACIONES)
VALUES (1, 'Brandon Ccalla', TIMESTAMP '2026-09-15 08:00:00', 500.00, 730.00, 'ABIERTA', 'Sesión de caja activa')

INSERT INTO MOVIMIENTOS_CAJA (ID, ID_SESION_CAJA, TIPO, MONTO, CONCEPTO, METODO_PAGO, REFERENCIA, FECHA)
VALUES (1, 1, 'INGRESO', 250.00, 'Venta mostrador ticket TKT-001', 'EFECTIVO', 'TKT-20260913-001', TIMESTAMP '2026-09-13 08:30:00')
INSERT INTO MOVIMIENTOS_CAJA (ID, ID_SESION_CAJA, TIPO, MONTO, CONCEPTO, METODO_PAGO, REFERENCIA, FECHA)
VALUES (2, 1, 'EGRESO', 20.00, 'Caja chica cinta de embalaje', 'EFECTIVO', 'REC-0045', TIMESTAMP '2026-09-13 09:00:00')

-- 6. VENTAS FACTURADAS
INSERT INTO VENTAS (ID, NUMERO_TICKET, FECHA, ESTADO, METODO_PAGO, SUBTOTAL, IGV, TOTAL, CLIENTE_NOMBRE)
VALUES (1, 'TKT-20260913-001', TIMESTAMP '2026-09-13 08:30:00', 'REGISTRADA', 'EFECTIVO', 211.86, 38.14, 250.00, 'Cliente X')

INSERT INTO DETALLE_VENTAS (ID, ID_VENTA, ID_PRODUCTO, ID_LOTE, NOMBRE_PRODUCTO, PRECIO_UNITARIO, CANTIDAD, MONTO_TOTAL)
VALUES (1, 1, 1, 1, 'Quinua Blanca Real Orgánica 1kg', 14.50, 10, 145.00);
INSERT INTO DETALLE_VENTAS (ID, ID_VENTA, ID_PRODUCTO, ID_LOTE, NOMBRE_PRODUCTO, PRECIO_UNITARIO, CANTIDAD, MONTO_TOTAL)
VALUES (2, 1, 6, 3, 'Arándano Fresco Biloxi Clamshell 125g', 6.20, 17, 105.40);

COMMIT;
```



[PEGAR AQUÍ CAPTURA DE INSERCIÓN DML Y COMMIT EN SQL DEVELOPER]

Captura mostrando las filas insertadas y el mensaje "Confirmación terminada (COMMIT)".

3. Almacenamiento de Tablas

Pruebas de verificación de los datos rellenos en el entorno de Oracle SQL Developer mediante sentencias SELECT *.

3.1 Consulta de Catálogo: SELECT * FROM PRODUCTOS



[PEGAR AQUÍ CAPTURA DE: SELECT * FROM PRODUCTOS;]

Cuadrícula de datos con ID, NOMBRE, PRECIO, STOCK, ID_CATEGORIA.

3.2 Consulta de Lotes: SELECT * FROM LOTES_PRODUCTO



[PEGAR AQUÍ CAPTURA DE: SELECT * FROM LOTES_PRODUCTO;]

Cuadrícula mostrando CODIGO_LOTE, STOCK_DISPONIBLE, FECHA_PRODUCCION y FECHA_VENCIMIENTO.

3.3 Consulta de Ventas: SELECT * FROM VENTAS



[PEGAR AQUÍ CAPTURA DE: SELECT * FROM VENTAS;]

Cuadrícula con NUMERO_TICKET, METODO_PAGO, SUBTOTAL, IGV, TOTAL, CLIENTE_NOMBRE.

3.4 Consulta de Detalle: SELECT * FROM DETALLE_VENTAS



[PEGAR AQUÍ CAPTURA DE: SELECT * FROM DETALLE_VENTAS;]

Cuadrícula con ID_VENTA, ID_PRODUCTO, ID_LOTE, CANTIDAD, SUBTOTAL.

4. Almacenamiento de Procedimientos (Stored Procedures - 2 SP)

Desarrolle e implemente 2 Stored Procedures requeridos para la lógica del sistema agroindustrial con control de concurrencia y validaciones de stock.

4.1 Stored Procedure 01: Registrar Venta con Control de Stock Atómico (RN-VNT-01)

Regla RN-VNT-01: Antes de registrar el ticket y detalle, se bloquea el registro del producto con FOR UPDATE. Si el stock en catálogo o lote no abastece la cantidad solicitada, lanza la excepción personalizada -20010 evitando sobreventas.

```

CREATE OR REPLACE PROCEDURE SP_REGISTRAR_VENTA_ITEM (
    p_cliente_nombre IN VARCHAR2,
    p_metodo_pago     IN VARCHAR2,
    p_id_producto     IN NUMBER,
    p_id_lote         IN NUMBER DEFAULT NULL,
    p_cantidad        IN NUMBER,
    p_numero_ticket   OUT VARCHAR2,
    p_total_cobrado   OUT NUMBER
) AS
    v_stock_actual    NUMBER(10);
    vPrecioProd       NUMBER(10, 2);
    v_nombre_prod     VARCHAR2(120);
    v_subtotal_item   NUMBER(12, 2);
    v_igv             NUMBER(12, 2);
    v_base_neta       NUMBER(12, 2);
    v_id_venta        NUMBER;
BEGIN
    -- Validar canal de cobro
    IF UPPER(p_metodo_pago) NOT IN ('EFFECTIVO', 'YAPE', 'PLIN', 'TRANSFERENCIA') THEN
        RAISE_APPLICATION_ERROR(-20011, 'Método de pago no válido.');
```

```

    END IF;

    -- Bloqueo pesimista concurrente
    SELECT STOCK, PRECIO, NOMBRE
    INTO v_stock_actual, vPrecioProd, v_nombre_prod
    FROM PRODUCTOS
    WHERE ID = p_id_producto
    FOR UPDATE;

    -- Validación RN-VNT-01
    IF v_stock_actual < p_cantidad THEN
        RAISE_APPLICATION_ERROR(-20010,
            'Regla RN-VNT-01: Stock insuficiente para ' || v_nombre_prod ||
            '. Disponible: ' || v_stock_actual || ' unidades.');
```

```

    END IF;

    -- Cálculos comerciales
    v_subtotal_item := vPrecioProd * p_cantidad;
    v_igv := ROUND(v_subtotal_item * 0.18, 2);
    v_base_neta := v_subtotal_item - v_igv;
    p_numero_ticket := 'TKT-' || TO_CHAR(SYSDATE, 'YYYYMMDDHH24MISS');
    p_total_cobrado := v_subtotal_item;

    -- Inserción en cabecera
    INSERT INTO VENTAS (NUMERO_TICKET, FECHA, ESTADO, METODO_PAGO, SUBTOTAL, IGV, TOTAL, CLIENTE_NOMBRE)
    VALUES (p_numero_ticket, SYSDATE, 'REGISTRADA', UPPER(p_metodo_pago), v_base_neta, v_igv, v_subtotal_item, p_cliente_nombre)
    RETURNING ID INTO v_id_venta;

    -- Inserción en detalle
    INSERT INTO DETALLE_VENTAS (ID_VENTA, ID_PRODUCTO, ID_LOTE, NOMBRE_PRODUCTO, PRECIO_UNITARIO, CANTIDAD,
    VALUES (v_id_venta, p_id_producto, p_id_lote, v_nombre_prod, vPrecioProd, p_cantidad, v_subtotal_item);

    -- Actualización de inventario físico
    UPDATE PRODUCTOS
    SET STOCK = STOCK - p_cantidad
    WHERE ID = p_id_producto;

    IF p_id_lote IS NOT NULL THEN
        UPDATE LOTES_PRODUCTO
        SET STOCK_DISPONIBLE = STOCK_DISPONIBLE - p_cantidad
        WHERE ID = p_id_lote;

```

```
END IF;  
  
COMMIT;  
EXCEPTION  
  WHEN OTHERS THEN  
    ROLLBACK;  
    RAISE;  
END;  
/
```



[PEGAR AQUÍ CAPTURA DE COMPILACIÓN DEL SP 01 EN SQL DEVELOPER]

Procedimiento SP_REGISTRAR_VENTA_ITEM compilado con éxito.

Prueba de Ejecución del SP 01:

```
-- Bloque anónimo de prueba: Registrar venta de 5 unidades de Quinoa Blanca
DECLARE
  v_ticket VARCHAR2(30);
  v_total  NUMBER;
BEGIN
  SP_REGISTRAR_VENTA_ITEM(
    p_cliente_nombre => 'Cooperativa Sol del Valle',
    p_metodo_pago    => 'EFECTIVO',
    p_id_producto    => 1,
    p_id_lote        => 1,
    p_cantidad       => 5,
    p_numero_ticket  => v_ticket,
    p_total_cobrado  => v_total
  );
  DBMS_OUTPUT.PUT_LINE('Venta registrada exitosamente.');
```

```
DBMS_OUTPUT.PUT_LINE('Ticket: ' || v_ticket || ' | Total Cobrado: S/ ' || v_total);
END;
/
```



[PEGAR AQUÍ CAPTURA DE EJECUCIÓN Y PRUEBA DEL SP 01]

Ventana de Salida DBMS mostrando el mensaje de venta exitosa y descuento de stock.

4.2 Stored Procedure 02: Anulación de Venta con Reposición Automática de Inventario

```
CREATE OR REPLACE PROCEDURE SP_ANULAR_VENTA (  
    p_id_venta IN NUMBER  
) AS  
    v_estado_actual VARCHAR2(20);  
BEGIN  
    SELECT ESTADO  
    INTO v_estado_actual  
    FROM VENTAS  
    WHERE ID = p_id_venta  
    FOR UPDATE;  
  
    IF v_estado_actual = 'ANULADA' THEN  
        RAISE_APPLICATION_ERROR(-20015, 'La venta ya se encuentra anulada previamente.');
```

-- Devolución de existencias al almacén

```
    FOR item IN (SELECT ID_PRODUCTO, ID_LOTE, CANTIDAD FROM DETALLE_VENTAS WHERE ID_VENTA = p_id_venta) LOOP  
        UPDATE PRODUCTOS  
        SET STOCK = STOCK + item.CANTIDAD  
        WHERE ID = item.ID_PRODUCTO;  
  
        IF item.ID_LOTE IS NOT NULL THEN  
            UPDATE LOTES_PRODUCTO  
            SET STOCK_DISPONIBLE = STOCK_DISPONIBLE + item.CANTIDAD  
            WHERE ID = item.ID_LOTE;  
        END IF;  
    END LOOP;  
  
    -- Asentar estado anulado  
    UPDATE VENTAS  
    SET ESTADO = 'ANULADA'  
    WHERE ID = p_id_venta;  
  
    COMMIT;  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        ROLLBACK;  
        RAISE_APPLICATION_ERROR(-20016, 'No existe la venta indicada.');
```

WHEN OTHERS THEN

```
        ROLLBACK;  
        RAISE;  
END;  
/
```



[PEGAR AQUÍ CAPTURA DE COMPILACIÓN Y PRUEBA DEL SP 02]

Ejecución de EXEC SP_ANULAR_VENTA(1); y comprobación del estado ANULADA.

5. Funciones Definidas por el Usuario (Functions - 2 Funciones)

Desarrolle e implemente 2 funciones personalizadas aplicadas al cálculo o reporte del sistema.

5.1 Función 01: Determinar Lote Prioritario por Vencimiento (Criterio FEFO)

Retorna el CODIGO_LOTE que debe salir primero del almacén para garantizar que el producto no caduque en anaquel:

```
CREATE OR REPLACE FUNCTION FN_OBTENER_LOTE_FEFO (  
    p_id_producto IN NUMBER,  
    p_cantidad    IN NUMBER  
) RETURN VARCHAR2 AS  
    v_codigo_lote VARCHAR2(30);  
BEGIN  
    SELECT CODIGO_LOTE  
    INTO v_codigo_lote  
    FROM (  
        SELECT CODIGO_LOTE  
        FROM LOTES_PRODUCTO  
        WHERE ID_PRODUCTO = p_id_producto  
              AND STOCK_DISPONIBLE >= p_cantidad  
        ORDER BY FECHA_VENCIMIENTO ASC  
    )  
    WHERE ROWNUM = 1;  
  
    RETURN v_codigo_lote;  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        RETURN 'SIN_LOTE_DISPONIBLE';  
END;  
/
```

Prueba de la Función 01:

```
SELECT  
    p.NOMBRE AS PRODUCTO,  
    FN_OBTENER_LOTE_FEFO(p.ID, 10) AS LOTE_PRIORITARIO_FEFO  
FROM PRODUCTOS p  
WHERE p.ID IN (1, 6);
```



[PEGAR AQUÍ CAPTURA DE COMPILACIÓN Y PRUEBA DE LA FUNCIÓN 01]

Resultado de la consulta mostrando el lote que vence más pronto (ej. LOT-ARA-2026-01).

5.2 Función 02: Reporte de Total Recaudado por Método de Pago

Totaliza el volumen de facturación en soles acumulado para un determinado método de pago en ventas no anuladas:

```
CREATE OR REPLACE FUNCTION FN_TOTAL_VENTAS_METODO (  
    p_metodo_pago IN VARCHAR2  
) RETURN NUMBER AS  
    v_total NUMBER(12, 2) := 0;  
BEGIN  
    SELECT NVL(SUM(TOTAL), 0)  
    INTO v_total  
    FROM VENTAS  
    WHERE UPPER(METODO_PAGO) = UPPER(p_metodo_pago)  
    AND ESTADO = 'REGISTRADA';  
  
    RETURN v_total;  
END;  
/
```

Prueba de la Función 02:

```
SELECT  
    FN_TOTAL_VENTAS_METODO('EFECTIVO') AS RECAUDADO_EFECTIVO,  
    FN_TOTAL_VENTAS_METODO('YAPE') AS RECAUDADO_YAPE,  
    FN_TOTAL_VENTAS_METODO('TRANSFERENCIA') AS RECAUDADO_TRANSFERENCIA  
FROM DUAL;
```



[PEGAR AQUÍ CAPTURA DE COMPILACIÓN Y PRUEBA DE LA FUNCIÓN 02]

Cuadrícula con los importes totalizados por canal de cobranza.